SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

Customer Relationship Management System

Authors: Shweta, Avaneesh N, Vishwas V Angadi, Krishna Pai
Version 2.2 | 04-Nov-2026 | Status: Revised Draft

Document Control

Field	Details
Project	Customer Relationship Management (CRM)
Document	Software Architecture and Design Specification
Authors	Shweta, Avaneesh N, Vishwas V Angadi, Krishna Pai
Scope baseline	GitHub Project Management issues visible in the supplied screenshots (#6–#34)
Status	Revised Draft — team review and instructor approval pending

Approvals

Role	Name
Team Member	Shweta
Team Member	Avaneesh N
Team Member	Vishwas V Angadi
Team Member	Krishna Pai
Instructor / Reviewer	To be completed

1. Introduction

1.1 Purpose

This document describes the architecture and design of the CRM application. It is aligned with the issue-derived SRS and Test Plan and documents the architecture, components, data responsibilities, security design, representative UML workflows and API areas.

1.2 Scope

The design covers authentication/user management, lead and customer management, customer engagement and support, sales and campaigns, dashboards/reports, audit logging, repository automation, testing/documentation and the associated non-functional requirements.

1.3 Audience

Developers, testers, project members, instructor/evaluators and maintainers.

1.4 Definitions

Term	Meaning
RBAC	Role-Based Access Control

API	Application Programming Interface
CI	Continuous Integration
SRS / STP	Software Requirements Specification / Software Test Plan
UML	Unified Modeling Language

2. Document Overview

2.1 How to Use this Document

Section 3 describes the system architecture and its rationale. Section 4 presents representative design workflows, API responsibilities, error handling and UX considerations. Diagrams are placed on separate pages at readable size.

2.2 Related Documents

- CRM Software Requirements Specification (SRS), Version 2.2.
- CRM Software Test Plan (STP), Version 2.2.
- GitHub CRM Project Management issue board and source repository.

3. Architecture

3.1 Goals and Constraints

- Separate presentation, backend business logic and persistence responsibilities.
- Enforce authentication and role authorization for protected pages and APIs.
- Support the CRM functions defined in the SRS.
- Preserve CRM records reliably.
- Maintain a modular structure and safe error handling.
- Use the configured GitHub Actions and pull-request checks.

3.2 Stakeholders and Concerns

Stakeholder	Primary Concern
Admin	User account administration and permitted oversight.
Sales Staff	Lead/customer workflows and permitted sales activities.
Support Staff	Permitted customer engagement and support operations.
Developers	Maintainability, modularity, API consistency and persistence.
Testers	Functional, security, performance and regression verification.
Instructor / Evaluator	Traceability and consistency across SRS, SAD and STP.

3.3 Component (UML) Diagram

The CRM follows a layered web architecture. The frontend communicates with the backend API. Backend routes and middleware apply authentication, authorization and validation before interacting with the database. GitHub Actions performs repository checks as configured.

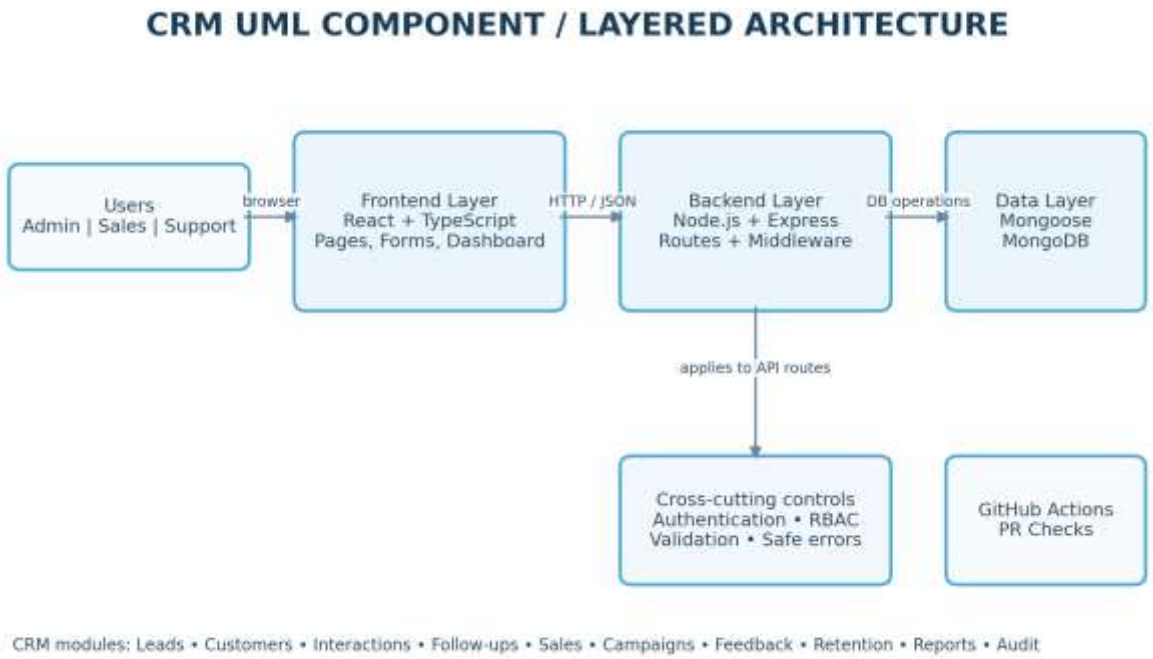


Figure 1. UML component / layered architecture diagram.

3.4 Component Descriptions

Component	Responsibility
Frontend UI	Authentication screens, navigation, forms, lists, dashboards and reports.
Authentication / Authorization	Credential validation, authentication state and role checks.
User Management	Authorized staff account management.
Lead / Customer modules	Lead creation and updates, lead conversion, customer profiles and assignment.
Engagement modules	Interaction tracking, follow-ups, feedback and retention.
Sales / Campaign modules	Sales records and campaign management.
Dashboard / Reports / Audit	Summary views, reports and relevant audit records.
Backend API / Middleware	Request handling, validation, role checks and safe error handling.
Database layer	Persistent storage and retrieval of CRM records.
CI workflow	GitHub Actions and pull-request checks.

3.5 Chosen Architecture Pattern and Rationale

A layered, modular web architecture is suitable for this academic CRM. The frontend is responsible for presentation, backend APIs enforce business and access rules, and the database layer manages persistence. This separation supports the modularity objective and allows independent functional and integration testing. A microservices architecture is not required by the current project scope.

3.6 Technology Stack and Data Stores

Layer	Technology / Responsibility
Frontend	React and TypeScript web interface.
Backend	Node.js and Express API layer.
Database	MongoDB with Mongoose models.
Repository Automation	GitHub Actions and pull-request checks.
Communication	Configured frontend/backend API requests and responses.

3.7 Risks and Mitigations

Risk	Mitigation
Unauthorized access	Enforce backend authentication and role checks.
Data loss or stale data	Verify persistence after refresh/restart.
Slow dashboard/list views	Measure representative performance and optimize bottlenecks.
Tightly coupled modules	Maintain clear module boundaries and review project structure.
Secret leakage	Use safe error responses and protect secrets in logs/responses.
Regression after changes	Run regression tests and configured PR checks.

3.8 Traceability to Requirements

SRS Requirement Area	Design Component	Related Project Work
CRM-F-001 to CRM-F-004; CRM-SR-001 to SR-003	Authentication and User Management	Login, validation, RBAC and account management.
CRM-F-005 to CRM-F-008	Lead / Customer modules	Lead lifecycle, conversion, profile and assignment.
CRM-F-009 to CRM-F-012	Engagement modules	Interactions, follow-ups, feedback and retention.
CRM-F-013 to CRM-F-016	Sales, Campaign, Dashboard, Audit	Business operations and oversight.
CRM-F-017 to CRM-F-019	CI, testing and documentation	GitHub Actions, regression and documentation.

CRM-NF-001 to CRM-NF-005; CRM-SR-004 to SR-005	Cross-cutting controls	Performance, persistence, modularity, safe errors and access protection.

3.9 Security Architecture

The backend is the authoritative security boundary. Authentication identifies the user and RBAC determines whether the user may perform an operation. The frontend may hide unavailable navigation items for usability, but every protected API must independently enforce the relevant permission. Input validation and safe error handling reduce invalid operations and information leakage.

Security Concern	Design Control
Unauthenticated access	Require authentication for protected pages and API operations.
Role misuse	Check the authenticated user's role before protected operations.
Credential/secret exposure	Validate credentials and avoid returning/logging sensitive values.
Information leakage	Return safe errors without stack traces or secrets.
Unauthorized record modification	Check permissions and record ownership before changes.

4. Design

4.1 Design Overview

A user action is initiated in the frontend. The API receives the request, verifies authentication and role permissions, validates the data and calls the relevant business module. The data layer performs the required read or write. The API returns a result that the frontend displays.

4.2 UML Sequence Diagrams

The sequence diagrams illustrate representative workflows for authentication, lead conversion and sales/report refresh.

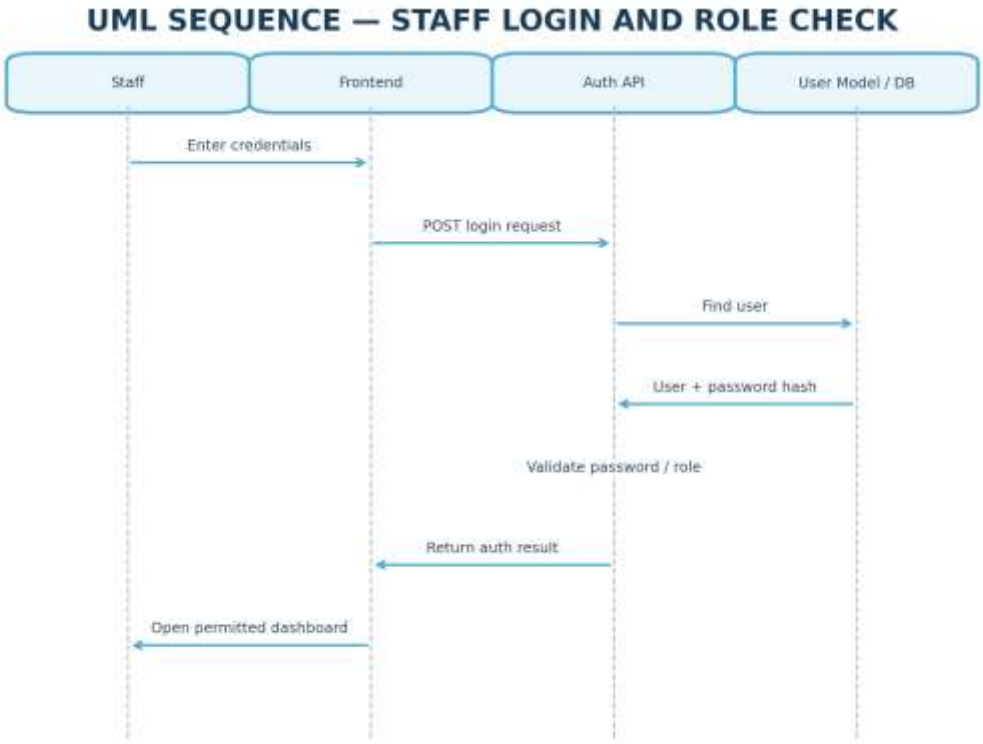


Figure 2. UML sequence diagram — staff login and role check.

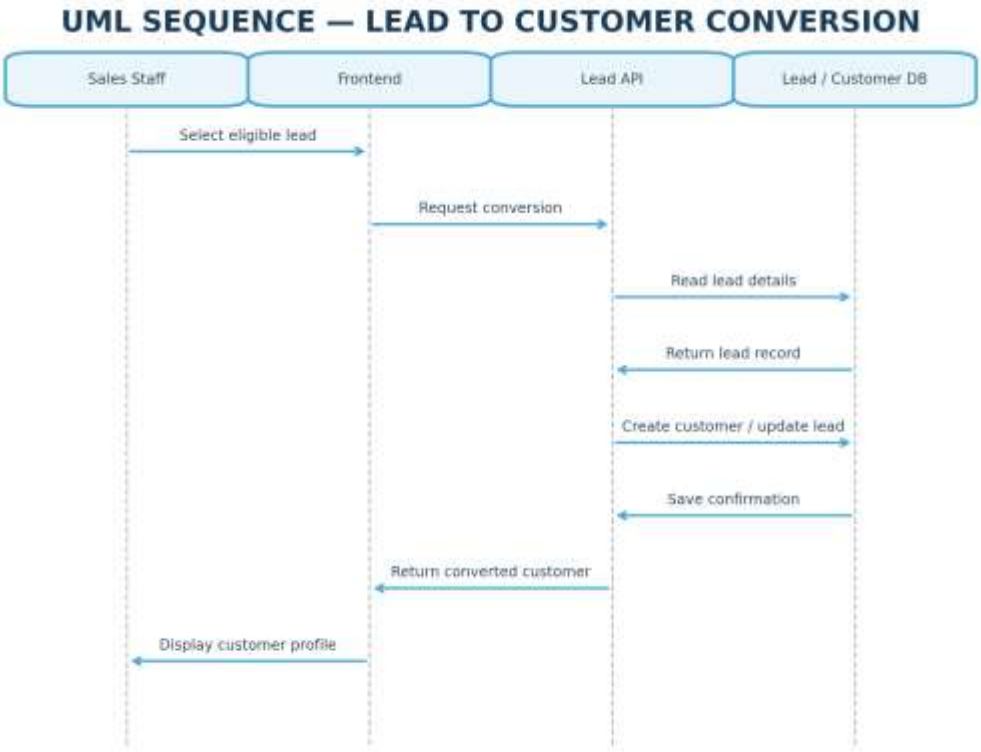


Figure 3. UML sequence diagram — lead-to-customer conversion.

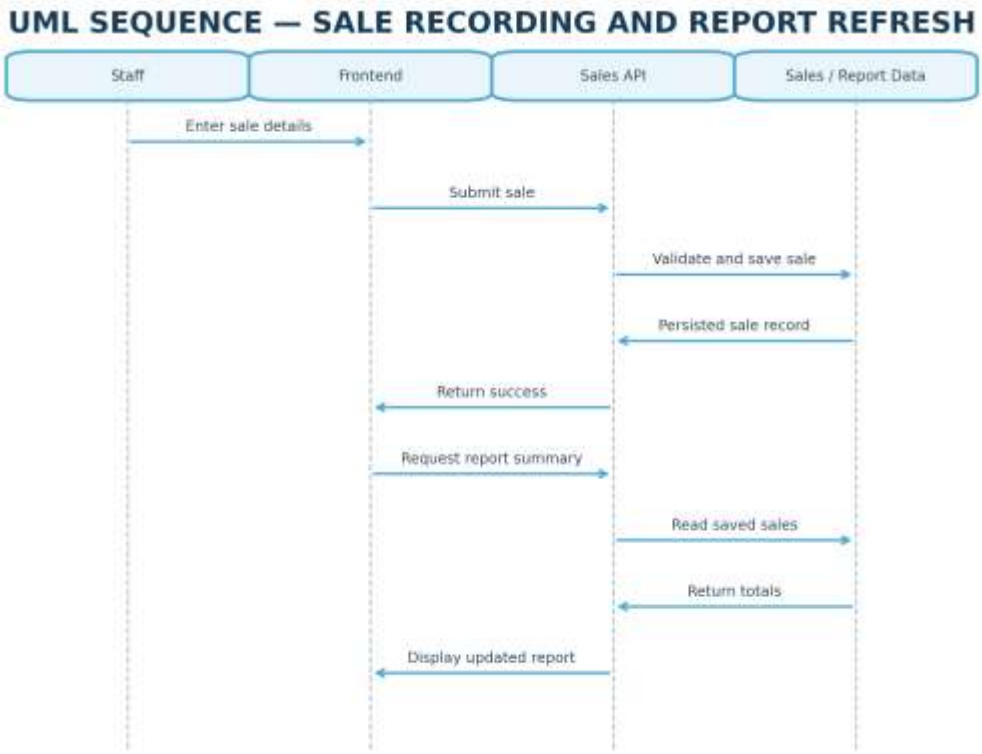


Figure 4. UML sequence diagram — sale recording and report refresh.

4.2.1 Conceptual Data Model

The conceptual model shows the main CRM records and their relationships. It is a high-level view; exact field names and cardinalities are defined by the final Mongoose schemas.

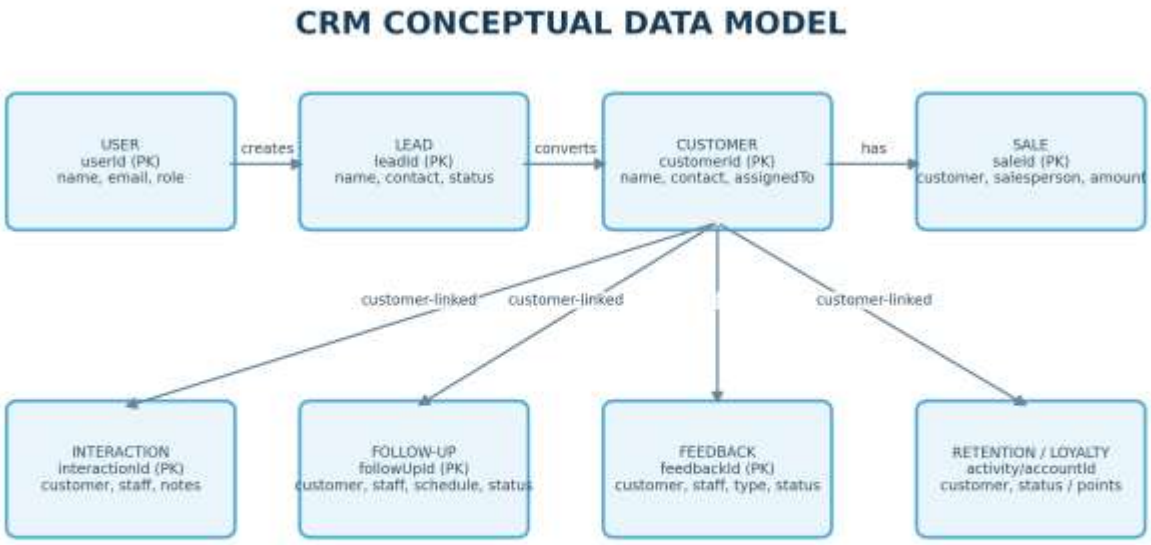


Figure 5. CRM conceptual data model.

4.3 API Design

The API is organized around the project's feature areas. This logical interface summary avoids inventing route paths or methods not verified from the final source code.

API / Module Area	Responsibility
Authentication / Users	Login/logout, credential checks, user accounts and role enforcement.
Leads / Customers	Lead management, conversion, customer profiles and assignment.
Interactions / Follow-ups	Customer communication history and follow-up lifecycle.
Feedback / Retention	Customer feedback and retention activities.
Sales / Campaigns	Sales records and campaign management.
Dashboard / Reports	Dashboard summaries and report data.
Audit Logging	Record and review relevant application events.

4.3.1 Common API Behavior

- Protected operations require authentication and role authorization.
- Input must be validated before database changes.
- Successful operations return an appropriate success response and result data.
- Invalid input, unauthenticated access and forbidden operations return safe error responses.
- Errors must not disclose passwords, tokens, secrets or internal stack traces.

4.4 Error Handling, Logging and Monitoring

The backend handles invalid input, authentication failures, forbidden operations, missing records and database failures using safe responses. Audit logging is part of the CRM scope. Performance verification measures dashboard and list loading. Sensitive credentials, tokens and secrets must not be exposed in responses or logs.

4.5 UX Design

- Provide clear login/logout and role-appropriate navigation.
- Use consistent forms and lists for CRM feature areas.
- Present dashboard/report information clearly.
- Show understandable validation and error messages without sensitive details.
- Maintain a consistent visual language across modules.

4.6 Open Issues and Next Steps

- Confirm exact API paths and request/response fields against the final source code before publishing an API contract.
- Record actual test results and attach evidence in the Test Plan.
- Document deviations between design and implementation if discovered during final review.
- Keep SRS, SAD and STP synchronized if project scope changes.

5. Appendices

5.1 Glossary

Term	Definition
RBAC	Role-Based Access Control.
CI	Continuous Integration.
API	Application Programming Interface.
Persistence	Saving data so it remains available after refresh/restart.
UML	Unified Modeling Language.